

SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1102

COURSE OUTCOME COVERED

CO4:

implement object-oriented system layers using design principles and patterns, and integrate access and view layers with OODBMS (*BL5*)

Assessment Tool Weightage: 40%

QUESTIONS: 5

Total Marks:40

Annexure A

Industry Problem Solving Task

Code of Conduct:

I E. Sakthi priya (192520041) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

E. Sakthi priya

Annexure A

Industry Problem Solving Task

1. Smart Banking and Financial Management System

A financial institution develops a **Smart Banking and Financial Management System** to manage customer accounts, fund transfers, loan processing, bill payments, and transaction history. The application follows a layered object-oriented architecture consisting of a presentation layer, service layer, domain layer, and data access layer connected to an OODBMS.

The system uses the **Factory Pattern** to create different account types, the **Strategy Pattern** for loan-interest and payment calculation policies, and the **Observer Pattern** for transaction and fraud alerts. A **Repository Pattern** is used to manage communication with persistent banking objects. The architecture is designed to support future services such as investment management and digital wallets.

Question:

Analyze how object-oriented principles, layered architecture, and design patterns contribute to the scalability, security, flexibility, and maintainability of the banking system. Explain how the application layers communicate with the OODBMS to store and retrieve persistent banking objects.

2. Smart Retail and Inventory Management System

A retail organization develops a **Smart Retail and Inventory Management System** to manage products, suppliers, customers, purchase orders, sales transactions, and warehouse stock. The system follows a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system applies the **MVC Pattern** for managing user interfaces, the **Factory Pattern** for creating different product categories, and the **Observer Pattern** to notify managers when inventory reaches a minimum threshold. The **DAO Pattern** handles communication with persistent inventory objects. The architecture allows future integration of automated stock prediction and robotic warehouse management.

Question:

Evaluate how object-oriented design principles, layered architecture, and design patterns improve the flexibility, scalability, and maintainability of the retail system. Explain how the different layers interact with the OODBMS for efficient inventory persistence and retrieval.

3. Smart Hotel and Hospitality Management System

A hotel chain develops a **Smart Hotel Management System** to manage guest registration, room reservations, check-in/check-out, payments, room services, and customer feedback. The application uses a layered object-oriented architecture consisting of presentation, service, domain, and data access layers connected to an OODBMS.

The system implements the **Strategy Pattern** for different pricing and discount policies, the **Factory Pattern** for creating room and service objects, and the **Observer Pattern** for reservation and service notifications. A **Repository Pattern** is used to simplify communication between business components and persistent hotel objects. Future enhancements include smart-room automation and personalized guest services.

Question:

Analyze how object-oriented principles and design patterns support flexibility, reusability, and maintainability in the hotel management system. Explain how the layered architecture coordinates reservation processing and communicates with the OODBMS for persistent data management.

4. Smart Logistics and Delivery Management System

A logistics company develops a **Smart Logistics and Delivery Management System** to manage shipment registration, warehouse operations, vehicle allocation, delivery tracking, and customer notifications. The system adopts a layered object-oriented architecture with presentation, business, domain, and data access layers connected to an OODBMS.

The system uses the **Strategy Pattern** for selecting optimal delivery routes, the **Factory Pattern** for creating different shipment types, and the **Observer Pattern** for real-time delivery status notifications. A **DAO Pattern** manages persistent shipment and vehicle information. The architecture is designed to support future integration with autonomous delivery vehicles and AI-based route optimization.

Question:

Examine how object-oriented design principles, layered architecture, and design patterns improve the adaptability, scalability, and maintainability of the logistics system. Explain how the layers communicate with the OODBMS to manage persistent shipment, vehicle, and delivery information.

5. Smart Energy Management System

An energy management company develops a **Smart Energy Management System** to monitor electricity consumption, manage smart meters, analyze energy usage, and generate alerts for

abnormal consumption. The system follows a layered object-oriented architecture where the presentation layer provides dashboards, the business layer performs energy analysis, the domain layer manages energy-related rules, and the data access layer communicates with an OODBMS.

The system uses the **Observer Pattern** for consumption alerts, the **Strategy Pattern** for different energy optimization algorithms, and the **Factory Pattern** for creating different smart-meter objects. A **Repository Pattern** is used to manage persistent energy data. The architecture is designed to accommodate future renewable-energy monitoring and AI-based consumption prediction.

Question:

Evaluate how object-oriented principles, layered architecture, and design patterns contribute to the scalability, extensibility, and maintainability of the energy management system. Explain the interaction between the system layers and the OODBMS for storing, retrieving, and managing persistent energy-monitoring objects.

RUBRICS FOR CALCULATION

Industry Problem Solving Task

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Industry Problem	20%	Comprehensive understanding of real-world problem and constraints	Good understanding with minor gaps in requirements	Basic understanding; some requirements unclear	Poor understanding ; misinterprets problem context
Application of Engineering Concepts	25%	Applies advanced and relevant concepts effectively to solve the problem	Applies appropriate concepts with minor errors	Limited application of concepts	Incorrect or no application of concepts
Solution Design	25%	Innovative, practical, and feasible solution aligned with industry standards	Logical and feasible solution with minor gaps	Basic solution with limited feasibility	Unclear or impractical solution
Use of Tools	15%	Effective use of modern tools, technologies, and real data	Appropriate use of tools with correct implementation	Limited or inconsistent use of tools	Minimal or incorrect use of tools
Analysis	10%	Thorough analysis with validated results and performance evaluation	Good analysis with reasonable validation	Basic analysis with limited validation	Poor or no analysis
Professionalism	5%	Highly professional, well-documented, and clearly presented work	Good documentation and presentation	Basic documentation with some gaps	Poor documentation and presentation

1. Smart Banking and Financial Management System

The Smart Banking and Financial Management System manages customer accounts, fund transfers, loans, bill payments and transaction history. It uses a layered object-oriented architecture consisting of the Presentation Layer, Service Layer, Domain Layer and Data Access Layer, connected to an OODBMS. It uses Factory, Strategy, Observer and Repository patterns.

Object-Oriented Principles

1. Encapsulation:

Banking data such as account balance, customer details and transaction information are kept inside appropriate classes. Access to sensitive data is controlled through methods.

2. Abstraction:

Complex operations such as fund transfer and loan processing are represented through simple service methods. The internal implementation is hidden from the user.

3. Inheritance:

Different account types such as SavingsAccount and CurrentAccount can inherit common properties from an Account class.

4. Polymorphism:

Different account or loan types can implement the same operation differently. For example, different loan types can have different interest calculations.

Layered Architecture

- **Presentation Layer:** Provides interfaces for customers and bank employees.
- **Service Layer:** Handles operations such as fund transfer, loan processing and bill payment.
- **Domain Layer:** Contains classes such as Customer, Account, Loan and Transaction.
- **Data Access Layer:** Uses repositories to store and retrieve persistent objects from the OODBMS.

This separation makes the system easier to modify and maintain.

Design Patterns

- **Factory Pattern:** Creates different account objects such as SavingsAccount and CurrentAccount without exposing object-creation logic.
- **Strategy Pattern:** Allows different loan-interest and payment calculation algorithms to be selected dynamically.
- **Observer Pattern:** Sends transaction and fraud alerts to registered observers whenever important transactions occur.
- **Repository Pattern:** Provides a common interface for storing, retrieving, updating and deleting banking objects from the OODBMS.

Security, Scalability and Maintainability

The layered architecture improves security because sensitive database operations are separated from the presentation layer. Encapsulation also protects banking data.

The system is scalable because new services such as investment management and digital wallets can be added without changing the complete system.

The use of design patterns improves maintainability and flexibility because individual components can be modified independently.

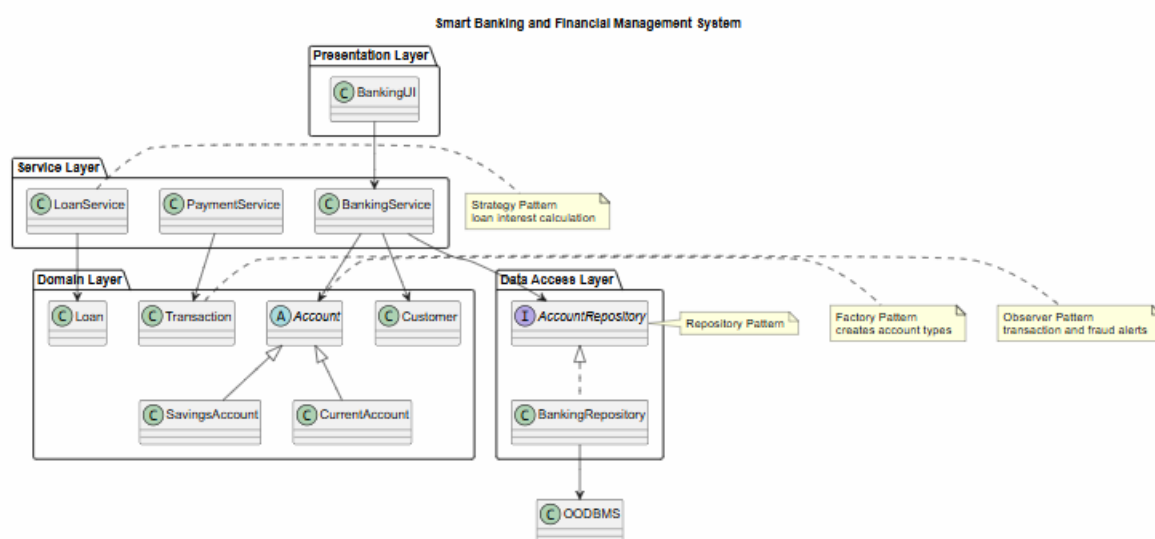
Communication with OODBMS

The flow is:

User → Presentation → Service → Domain → Repository/Data Access → OODBMS

For example, when a customer checks their balance, the presentation layer sends the request to the service layer. The service layer requests the required Account object from the repository. The data access layer retrieves the persistent object from the OODBMS and returns it through the layers to the presentation layer.

PlantUML – Banking Architecture



2. Smart Retail and Inventory Management System

The Smart Retail and Inventory Management System manages products, suppliers, customers, purchase orders, sales transactions and warehouse stock. It follows a layered architecture and uses MVC, Factory, Observer and DAO patterns.

Object-Oriented Principles

1. **Encapsulation:**
Product information, stock quantity and supplier information are maintained inside appropriate objects.
2. **Abstraction:**
Users interact with operations such as adding products or checking stock without knowing the internal database implementation.
3. **Inheritance:**
Different product categories can inherit common attributes from a Product class.
4. **Polymorphism:**
Different product categories can implement category-specific operations differently.

Layered Architecture

- **Presentation Layer:** Provides the retail user interface.
- **Business Layer:** Handles sales, purchases and inventory rules.
- **Domain Layer:** Contains Product, Supplier, Customer, Order and Stock objects.
- **Data Access Layer:** Communicates with the OODBMS through DAO objects.

Design Patterns

- **MVC Pattern:** Separates the user interface into Model, View and Controller components.
- **Factory Pattern:** Creates different product category objects.
- **Observer Pattern:** Monitors inventory levels and notifies managers when stock reaches the minimum threshold.
- **DAO Pattern:** Provides methods to store and retrieve persistent inventory objects from the OODBMS.

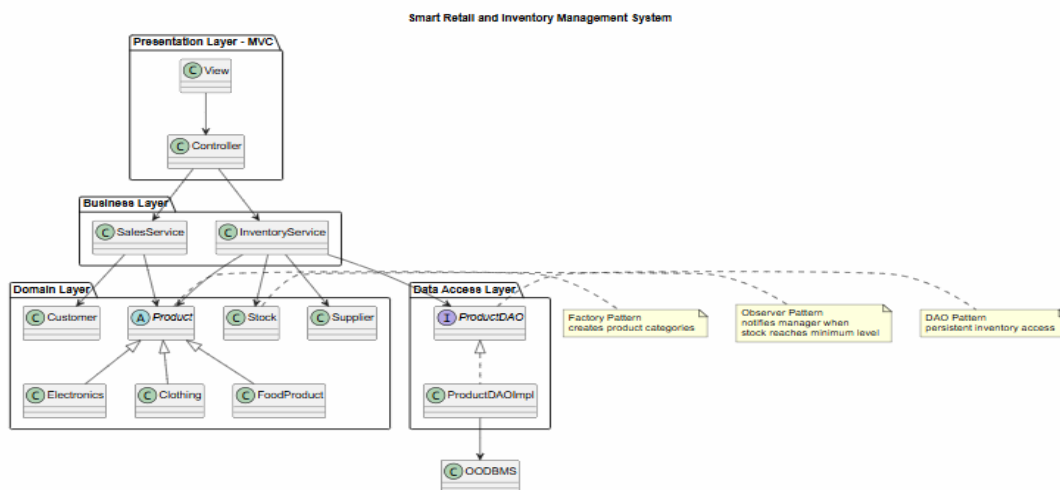
Flexibility and Scalability

- The system is flexible because new product categories can be added using the Factory Pattern without changing the existing code.
- It is scalable because future features such as automated stock prediction and robotic warehouse management can be integrated with the existing layers.
- The separation of presentation, business, domain and data access responsibilities improves maintainability.

OODBMS Interaction

For example, when a manager checks stock, the request goes from the presentation layer to the business layer. The business layer requests stock information from the DAO. The DAO retrieves Product and Stock objects from the OODBMS and returns them to the business layer.

PlantUML – Retail System



3. Smart Hotel and Hospitality Management System

The Smart Hotel Management System manages guest registration, room reservations, check-in/check-out, payments, room services and customer feedback. It uses Strategy, Factory, Observer and Repository patterns with a layered architecture connected to an OODBMS.

Object-Oriented Principles

1. **Encapsulation:**
Guest, room, reservation and payment information is maintained inside their respective classes.
2. **Abstraction:**
The customer can reserve a room without knowing how availability is checked internally.
3. **Inheritance:**
Different room types such as StandardRoom, DeluxeRoom and Suite can inherit from a common Room class.
4. **Polymorphism:**
Different room types can have different pricing and service behavior.

Design Patterns

- **Strategy Pattern:**
Different pricing and discount strategies can be selected depending on room type, season or customer category.
- **Factory Pattern:**
Creates different Room and Service objects.
- **Observer Pattern:**
Notifies guests or hotel staff about reservation and service status changes.
- **Repository Pattern:**
Provides a simplified interface for accessing persistent hotel objects.

Reservation Processing

A typical reservation process is:

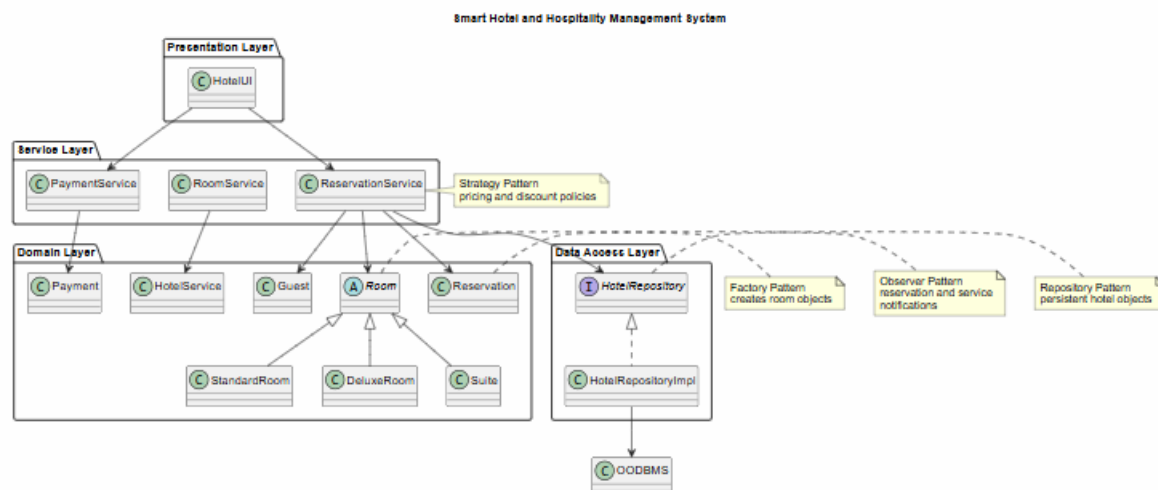
Guest → Presentation Layer → Reservation Service → Room/Reservation Domain Objects → Repository → OODBMS

The service layer checks room availability and applies the appropriate pricing strategy. After successful reservation, the reservation object is stored through the repository.

Maintainability and Reusability

- The layered design separates responsibilities, making the system easier to maintain.
- Design patterns make the system reusable. For example, a new discount strategy can be added without changing the reservation service.
- The architecture can also support future features such as smart-room automation and personalized guest services.

PlantUML – Hotel System



4. Smart Logistics and Delivery Management System

The Smart Logistics and Delivery Management System manages shipment registration, warehouse operations, vehicle allocation, delivery tracking and customer notifications. It uses Strategy, Factory, Observer and DAO patterns.

Object-Oriented Principles

- 1. Encapsulation:**
Shipment, vehicle and delivery information is stored within their respective objects.
- 2. Abstraction:**
Users can request shipment delivery without knowing the internal route-selection process.
- 3. Inheritance:**
Different shipment types can inherit from a common Shipment class.
- 4. Polymorphism:**
Different delivery strategies can calculate routes differently depending on the shipment and delivery requirements.

Design Patterns

- **Strategy Pattern:**
Used to select different delivery-route algorithms. For example, the system can select the fastest or shortest route.
- **Factory Pattern:**
Creates different shipment types.
- **Observer Pattern:**
Provides real-time delivery status notifications to customers or administrators.
- **DAO Pattern:**
Manages storage and retrieval of persistent shipment and vehicle information.

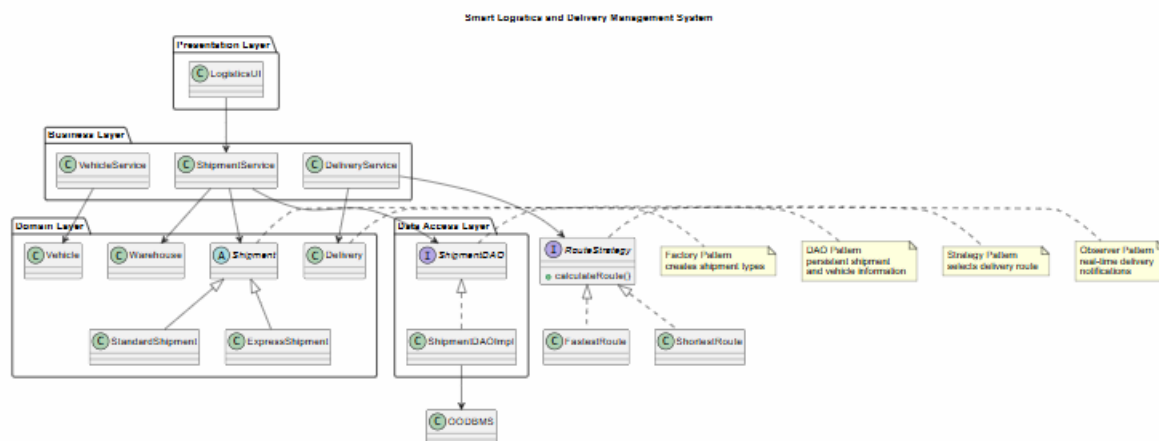
Layer Interaction

- The user registers a shipment through the presentation layer. The business layer processes the shipment request and selects an appropriate route strategy. Domain objects represent shipments, vehicles and deliveries.
- The DAO communicates with the OODBMS to store and retrieve these objects.

Adaptability and Scalability

- The system can easily support new shipment types using the Factory Pattern.
- New route-selection algorithms can be introduced using the Strategy Pattern without modifying the main delivery service.
- The system can also be extended to support autonomous delivery vehicles and AI-based route optimization, as specified in the problem.

PlantUML – Logistics System



5. Smart Energy Management System

The Smart Energy Management System monitors electricity consumption, manages smart meters, analyzes energy usage and generates abnormal-consumption alerts. It uses Observer, Strategy, Factory and Repository patterns and communicates with an OODBMS through the data access layer.

Object-Oriented Principles

1. **Encapsulation:**
Energy consumption, meter readings and analysis information are maintained inside domain objects.
2. **Abstraction:**
Users can view energy consumption and optimization recommendations without knowing the internal processing.
3. **Inheritance:**
Different smart meter types can inherit from a common SmartMeter class.
4. **Polymorphism:**
Different energy optimization algorithms can provide different results while following the same interface.

Design Patterns

- **Observer Pattern:**
Monitors energy consumption and sends alerts when abnormal consumption is detected.
- **Strategy Pattern:**
Allows different energy optimization algorithms to be selected.
- **Factory Pattern:**
Creates different types of smart meter objects.
- **Repository Pattern:**
Manages storage and retrieval of persistent energy-monitoring objects.

Layered Architecture

- **Presentation Layer:** Provides dashboards for viewing energy consumption.
- **Business Layer:** Performs energy analysis.
- **Domain Layer:** Contains SmartMeter, EnergyReading, Consumption and Alert objects.
- **Data Access Layer:** Communicates with the OODBMS through the Repository.

OODBMS Interaction

- When a smart meter sends a reading, the data is converted into an EnergyReading domain object. The business layer analyzes the reading. The repository stores the persistent object in the OODBMS.
- When a user requests historical consumption, the repository retrieves the required EnergyReading objects from the OODBMS and returns them through the business and presentation layers.

Scalability and Extensibility

- The system is scalable because new smart-meter types and optimization algorithms can be added without changing the complete system.
- The Repository Pattern isolates database operations, while Strategy allows new optimization methods to be added easily.
- The architecture can support future renewable-energy monitoring and AI-based consumption prediction, as stated in the problem.

PlantUML – Energy Management System

